

Cache 与存储层次

怎样维护一个正确的高速副本

408 · 计算机组成原理 Topic CO-06

母问题：小而快的副本怎样证明“它就是那个数据”？

Cache 不是小型主存，而是带身份、有效性、修改状态和替换策略的副本系统：

Locality → Block Transfer → Placement → Tag / Valid → Offset → Replacement / Write State → AMAT

每次访问既要找到候选位置，又要证明候选行保存的是请求块，并决定副本修改何时向下传播。

本册学习范围

- 本册解释 Cache 为什么能加速访问、如何由地址找到候选行、怎样用标签确认身份，以及缺失时如何取回和替换数据。
- 本册重点解释局部性、块、组、标签、有效位、脏位、相联度、替换、写策略、3C 缺失和平均访存时间。
- 文件缓存、页表和 DMA 只作为可能的下一级或并行请求出现；Cache 的副本状态和正确性先在本册内闭合。

1 术语先行：Cache 不是“更快的主存”这么简单

本册的十个关键词

- **Cache**：容量较小但更快的存储副本；它保存下一级存储中部分数据的拷贝。
- **块/Cache line**：Cache 与下一级之间一次搬运的数据单位；块内偏移决定取哪一个字节。
- **组 set**：若干候选 Cache 行的集合；组索引决定访问哪一组。
- **标签 tag**：用来证明候选行对应哪个主存块的高位地址。
- **有效位 valid**：说明该行是否已经装入可用数据；无效行即使标签相同也不能命中。
- **脏位 dirty**：说明 Cache 行是否被修改而尚未写回下一级。
- **相联度**：每个组包含的行数；直接映射为 1 路，全相联把所有行看成一组。
- **命中/缺失 hit/miss**：命中表示身份和有效性都通过；缺失表示需要向下一级请求或先处理替换。
- **3C 缺失**：容量缺失、冲突缺失和强制缺失三种反事实分类。
- **AMAT**：平均访存时间，通常等于命中时间加“缺失率乘缺失代价”。

目录

1 术语先行：Cache 不是“更快的主存”这么简单	1
2 为什么“按地址直接找数据”不够	3
3 地址三分与组织	3
4 Hit/Miss 生命周期	3
4.1 Read hit	3
4.2 Read miss	3
4.3 3C 模型	3
5 写策略是两条独立轴	4
6 替换与状态成本	4
7 贯穿母例：两个块反复驱逐	4
8 AMAT 与物理位数	4
9 地址翻译接口	4
10 五列概念边界	5
11 做题控制协议	5
12 本册与存储层次的接口	5
13 知识地图：先证明副本身份，再计算性能	6
14 补充细节：副本正确性先于命中率	6
14.1 层次化存储器：局部性只说明为什么值得缓存	6
14.2 地址三分与容量位数	6
14.3 四种映射：定位候选与证明身份	7
14.4 替换算法与 Belady 边界	7
14.5 写策略是两条正交轴	7
14.6 完整 miss 生命周期与状态转换	7
14.7 3C 模型：分类必须绑定反事实基线	7
14.8 AMAT、效率与重叠	8
14.9 TLB/Cache 组合与 VIPT 边界	8
14.10 Cache 综合题的状态表	8

14.11 层次设计的三重矛盾与参数权衡	8
--------------------------------	---

2 为什么“按地址直接找数据”不够

Cache 容量小于下一级，多个主存块必须复用有限行。因此访问必须依次回答：允许放在哪？当前行属于哪个块？该块是否有效？块内取哪一字节？若被修改，谁拥有最新副本？这五问生成 placement、tag、valid、offset、dirty 和 replacement。

局部性是收益来源而非正确性证明：时间局部性预测近期重访，空间局部性预测邻近访问；预测失败仍必须通过 tag/valid 维持正确性。

3 地址三分与组织

设地址宽度为 m 位，数据容量为 C 字节，块大小为 B 字节，相联度为 E ，则组数

$$S = \frac{C}{B \times E}, \quad b = \log_2 B, \quad s = \log_2 S, \quad t = m - s - b.$$

地址字段为 ‘tag | set index | block offset’。若按字编址或 B 用字表示，先统一单位。

组织	候选位置	优点	代价
直接映射	1 行	命中路径短、无需替换选择	冲突风险高
E 路组相联	同组 E 行	成本与冲突折中	E 个 tag 比较器和选择器
全相联	任意行	映射冲突最少	全部 tag 并行比较、替换成本高

4 Hit/Miss 生命周期

4.1 Read hit

索引定位候选组，比较 ‘valid=1’ 且 tag 匹配的行，再由 offset 选择块内字节/字。命中只证明副本身份和有效性，不表示下一级没有其他副本。

4.2 Read miss

choose invalid/victim → write back if dirty → fetch block → fill tag/data/state → retry request.

顺序流水线可能 stall；能否继续执行由 CO-04 和实现的非阻塞结构决定。

4.3 3C 模型

compulsory 是第一次访问；capacity 是理想全相联同容量也装不下；conflict 是有限映射额外造成的互逐。分类必须绑定块大小、容量、相联度和访问序列，不能把所有 miss 都叫 “Cache 太小”。

5 写策略是两条独立轴

Hit 时: write-through 同时更新下一级, 流量大但一致关系直接; write-back 只更新 Cache 并设 dirty, 逐出时才写回, 流量较小但状态复杂。Miss 时: write-allocate 先取块再写入; no-write-allocate 绕过 Cache 写下一级。write-back 不自动推出 write-allocate。

6 替换与状态成本

直接映射没有选择; 组相联/全相联需在候选中选 victim, 可用 FIFO、Random、近似/精确 LRU。精确 LRU 的元数据和更新逻辑随相联度增长, 不能笼统断言“每行只需 $\log_2 E$ 位”。增大相联度降低冲突, 可能增加 hit time、功耗和比较器数量。

7 贯穿母例：两个块反复驱逐

对直接映射 Cache, 将地址除以 B 得 block number, 再算 $set = block \bmod S$ 。若两个 block 的 set 相同而 tag 不同, 交替访问会互相驱逐, 即使其他组为空。提高相联度让它们在同组共存, 但需要额外比较与 victim 选择。

典型错误路径

只看 tag 相同就判 hit, 漏了 valid; 只提高 Cache 容量就声称冲突一定消失; 只看 hit rate 就宣称更快, 漏了更长 hit time 或更大 miss penalty。

8 AMAT 与物理位数

单级近似:

$$AMAT = T_{hit} + MR \times MP.$$

多级时 miss penalty 可能包含下一级 AMAT、写回脏块和总线占用。使用公式必须声明 hit time 是否包含并行 tag/data、MP 是额外时间、写回成本是否计入、stall 是否可重叠。

每行物理位数至少为 data、tag、valid, write-back 再加 dirty, 替换/ECC 元数据按题设另算:

$$C_{physical} = N_{lines}(data + tag + valid + dirty + metadata).$$

“Cache 容量”若只指数据区, 不能直接拿物理总位数替代。

9 地址翻译接口

PIPT、VIPT、VIVT 是 Cache 使用 VA/PA 与 TLB 的组织选择。VIPT 能并行的关键是索引位不依赖被翻译改变的 VPN 位; 具体约束由页内偏移、块大小和相联度推出, 不背固定上限。Cache hit 也不等于 TLB hit。

10 五列概念边界

概念 A	≠ 概念 B	真正区别与题面信号	混淆后果
Cache	≠ 主存	小容量带身份副本 vs 原始层级存储	把 tag/valid 当普通地址位
Tag	≠ Index	身份证明 vs 候选组定位	地址三分错位
Valid	≠ Dirty	行内容是否可用 vs 是否比下一级更新	miss/写回条件错
Write-through	≠ Write-allocate	hit 更新传播轴 vs miss 是否取块轴	从一个策略推另一个
Hit rate	≠ Total time	命中比例 vs hit/miss 成本综合	只按命中率判快慢
Cache miss	≠ TLB miss	数据副本未命中 vs 翻译副本未命中	把重试处理器写错

11 做题控制协议

1. 统一编址单位、主存地址宽度、数据容量、块大小和相联度；
2. 算 line/set/offset/index/tag，并把访问转为 ‘(block,set,tag,offset)’；
3. 逐次推进 valid/tag/dirty/replacement 状态；
4. miss 写 victim、write-back、fill、策略和 retry；
5. AMAT 先声明成本模型，避免重复计算同一 penalty；
6. 用边界地址、冲突地址和替换后重访独立验证。

压缩成第一动作

看到 Cache 题，先问：**index 找到哪些候选？valid+tag 如何证明身份？offset 取哪一段？miss 时谁被替换、谁拥有最新数据？**

12 本册与存储层次的接口

从请求地址到副本状态

本册回答“目标块现在在哪个副本、怎样证明身份、缺失后向哪一级请求”，并给出填充、停顿和数据 ready 的时刻。下一级存储只需提供块和服务时间；地址翻译、流水线重叠等因素再作为题设条件接入。

Cache hit rate 不是完整 Cost；必须同时考虑 hit time、miss penalty、写回/填充流量、替换元数据与与流水线的重叠。它也不等于权限成功，更不等于 OS Page Cache 命中。

13 知识地图：先证明副本身份，再计算性能

考点	本册机制	接口
局部性/层次	block transfer、hit/miss、AMAT	CO-05
映射与替换	地址三分、3C、victim state	Rules
写策略/容量	dirty、写回、物理位数	CO-05/CO-04
TLB/Cache 组合	PIPT/VIPT/VIVT 最小边界	CO-07/CO-B02

一句话

Cache 的正确性来自“候选位置 + valid/tag 身份证明 + offset 定位 + 写状态管理”，性能才是在这个正确副本系统上由局部性、替换和 AMAT 产生的结果。

固定命中周期、精确 LRU 位数、所有缺失都阻塞整机等数值不是普遍常数，必须由题设给出。稳定方法是先完成地址分解、身份检查和副本状态转移，再把重叠或阻塞条件代入 AMAT。

14 补充细节：副本正确性先于命中率

Cache 的基本原理、映射、替换、写策略和性能问题都可以回接到一个副本状态机：

Request Address → Block / Set / Tag / Offset → Valid Identity Check → Hit or Victim Path → Fill / Write-back

Cache 的第一问题是“这个小副本是否能证明自己就是目标块”，第二问题才是“命中率有多高”。容量、tag、valid、dirty、替换元数据和一致性责任必须分层。

14.1 层次化存储器：局部性只说明为什么值得缓存

时间局部性意味着刚访问的数据可能再次访问，空间局部性意味着邻近地址可能很快被访问。层次化存储用更小更快的上层副本换取平均访问时间下降，但每层都存在容量、块粒度、命中时间、填充成本和一致性边界。Cache line、DRAM row、页和磁盘块可能大小不同，不能把一种局部性粒度直接当成所有层的传输粒度。

14.2 地址三分与容量位数

设主存地址宽度为 A 位、块大小为 $B = 2^b$ 字节、Cache 有 $S = 2^s$ 个组，则字节编址时

$$offset = b, \quad index = s, \quad tag = A - b - s.$$

若题目按字编址，先把块大小换成“每块多少个编址单元”，再重新计算 offset。直接映射每组一行，组相联每组 E 行，全相联只有一个候选组；“组数”和“行数”不能互换。

数据容量、行数和总物理位数分别为不同问题：

$$C_{data} = N_{lines} \times B \times 8,$$

而总物理位数还要加 tag、valid、dirty、替换位和题设 ECC。容量题若问“Cache 容量”必须确认是 data-only 还是含元数据。

14.3 四种映射：定位候选与证明身份

直接映射用 $set = block \bmod S$ 找唯一候选行，比较一次 tag；全相联把块放到任意行，需要比较所有有效 tag； E 路组相联先定位组，再在组内比较 E 个候选。映射选择是在“比较器/功耗/命中时间”和“冲突概率/硬件复杂度”之间取舍，不能只说相联度越大越好。

14.4 替换算法与 Belady 边界

直接映射没有 victim 选择；组相联/全相联在候选中按 FIFO、LRU、Random 或近似策略选出一行。精确 LRU 的元数据随相联度增加，不能笼统说每行只需 $\log_2 E$ 位；实际可能需要排序关系、计数器或近似树。FIFO/Random/LRU 的比较必须绑定访问序列和容量。

栈算法（如理想 LRU）满足容量增加时缺页/Cache miss 不增加，非栈算法可能出现 Belady anomaly；这一结论不能泛化到所有替换策略。颠簸通常意味着工作集在当前映射/容量下反复互逐，改善方法可能是提高容量、相联度、块大小或改变访问顺序，但每种方法都有 hit time、带宽和污染代价。

14.5 写策略是两条正交轴

命中时的传播策略和未命中时的取块策略必须分开：

- write-through：命中时同时更新下一级，状态直接但写流量大；
- write-back：只更新 Cache 并置 dirty，逐出时写回，流量小但状态复杂；
- write-allocate：写 miss 先取入整块，再在 Cache 中修改；
- no-write-allocate：写 miss 绕过 Cache，直接写下一级。

常见搭配是 write-back + write-allocate 或 write-through + no-write-allocate，但一种轴绝不自动决定另一轴。多核环境下还要增加一致性/写回责任，不能把单核教材策略直接当作完整一致性协议。

14.6 完整 miss 生命周期与状态转换

一次 read miss 的最小链是

detect $\rightarrow$ choose invalid/victim $\rightarrow$ write back if dirty $\rightarrow$ fetch block $\rightarrow$ fill data/tag/valid $\rightarrow$ retry request.

写 miss 还要插入 write-allocate/no-write-allocate 分支。填充顺序必须先把数据写入行，再使 valid/tag 对后续访问可见；若题目涉及并发或非阻塞 Cache，还要标出 MSHR/等待者和数据 ready 时刻。Cache miss 可能使当前流水线停顿，也可能允许无关指令继续执行，属于 CO-04 的时序接口。

14.7 3C 模型：分类必须绑定反事实基线

compulsory miss 是第一次访问块，capacity miss 是即使理想全相联、同容量也装不下，conflict miss 是有限映射额外造成的互逐。分类不是从 miss 的表面现象猜，而是比较不同容量/相联度基线下的访问序列。块大小改变后，原来的 compulsory、capacity 和 conflict 归类可能随工作集和污染改变。

14.8 AMAT、效率与重叠

单级近似为

$$AMAT = T_{hit} + MR \times MP.$$

多级结构中 MP 可能包含下一级 AMAT、脏 victim 写回、总线仲裁和填充；若 hit time 或 miss penalty 已包含某部分，不得重复计算。题目还可能给出流水线 stall、并行 tag/data 查找或非阻塞重叠，此时先写事件时间线，再判断哪些项相加、哪些取最大。

加速比和效率也要绑定基准：命中率提高但 hit time 增大，或 miss penalty 下降但 IC/CPI 改变，都可能让完整 CPU time 的比较结果不同。不能只凭命中率或 AMAT 中一个局部参数宣告系统更快。

14.9 TLB/Cache 组合与 VIPT 边界

PIPT 先用物理地址查 TLB 后查 Cache，身份直接但路径可能串行；VIVT 用虚拟地址查 Cache，可能遇到 synonym/homonym；VIPT 用页内偏移中的 index 并行查 Cache 与 TLB，再用翻译后的 tag 确认身份。VIPT 能否不发生额外别名约束取决于

$$S \times B \leq \text{page size}$$

所对应的索引是否完全落在页内偏移中；必须结合相联度、页大小和地址位数推导，不能背一个固定容量上限。Cache hit 不等于 TLB hit，TLB miss、Page Fault 和 Cache miss 的处理者与重试点不同。

14.10 Cache 综合题的状态表

状态字段	解释	改变事件	验证问题
valid	行是否包含可用副本	首次 fill、失效、替换	tag 相同但 valid=0 能否命中？
tag	副本身份	fill/替换	当前 block 是否与 tag 一致？
dirty	Cache 是否比下一级新	write-back 命中写入、写回	victim 是否必须先写回？
replacement	组内选择历史	每次访问/替换	策略与访问序列是否一致？
pending/ready	miss 请求与返回时间	下一级响应、重试	当前流水线何时可继续？

CO-06 的陌生题复原

先统一编址单位、块大小、组数、相联度和地址宽度；再算 ‘(block,set,tag,offset)’，逐访问更新 valid/tag/dirty/replacement，最后写 victim、fill、write-back 和 retry。只有这条状态链闭合，AMAT 和容量位数才有可靠输入。

14.11 层次设计的三重矛盾与参数权衡

存储系统同时追求速度快、容量大、每位成本低；SRAM、DRAM、磁盘/SSD 分别在这些轴上占优，单一介质无法同时满足。寄存器虽然位于存储阶梯最上方，但由指令显式指定，不经过 hit/miss、映射和

替换，因此不应机械地当作 Cache 层次。真正的缓存关系至少包括 Cache-主存和主存-后备存储：前者由硬件管理、以块为单位，后者由 OS 管理、以页为单位。

两个层次的结构不意味着参数相同。缺页代价远高于 Cache miss，故虚存倾向更灵活的映射、软件参与和写回；Cache 则用较快的硬件查找、有限相联度和可选 write-through。主存/后备存储的平均访问时间可在串联模型下写成

$$T_{avg} = H_c T_c + (1 - H_c) T_{miss,c}, \quad T_{miss,c} = T_m + (1 - H_m) T_d,$$

但若题目采用“先查 Cache，miss 后才启动主存”的模型，应改写为 $T_c + (1 - H_c) T_m$ ，并明确 T_d 是否已经包含 OS/设备处理。

块大小、页大小、Cache 容量和相联度都不是越大越好：块/页变大可利用更多空间局部性、摊薄启动成本，却会减少候选块数、增加污染、搬运时间和内部碎片；容量增大提高命中率，却可能提高命中延迟和成本；相联度增大减少冲突，却增加比较器、功耗和 tag 路径延迟。页通常大于 Cache 块，是因为磁盘/SSD 的启动与访问代价远高于主存电路延迟，而不是因为“页一定更重要”。

层次题的停止条件

先说明对象属于寄存器、Cache、主存还是后备存储，再写本层的管理者、传输单位、映射与 miss 处理者；最后声明块/页大小、命中率和 miss penalty 的时间模型。若答案只说“层次越多越快”，却没有指出局部性和代价分布，推理尚未闭合。